

# Lab Exercise: Cookies and Cache-Control in FastAPI

**Duration:** ~60–90 minutes

**Objective:**

- Learn how to set, read, and delete cookies in FastAPI.
- Learn how to add caching headers ( `Cache-Control` , `ETag` ) to responses.
- Practice researching documentation and applying it to your own API.

## Part 1: Cookies

### 1. Set a Cookie

- Create an endpoint (e.g., `/login/{username}`) that sets a cookie with the username.
- Hint: Look up how to use `Response` in FastAPI to set cookies.

### 2. Read a Cookie

- Create an endpoint (e.g., `/profile`) that reads the cookie and greets the user.
- Hint: FastAPI has a `Cookie` dependency you can use.

### 3. Delete a Cookie

- Create an endpoint (e.g., `/logout`) that deletes the cookie.
- Hint: There's a method on `Response` for this.

**Test:**

- Use `curl -i` or your browser's developer tools to check that the cookie is being set, sent back, and deleted.

## Part 2: Cache-Control

### 1. Add Cache-Control Header

- Modify your `/questions` (or `/devices`) endpoint to include a `Cache-Control` header.
- Hint: You can set headers on the `Response` object.

### 2. Add ETag Support (Optional, Advanced)

- Generate a simple hash of your response data.
- Send it as an `ETag` header.
- If the client sends `If-None-Match` with the same value, return `304 Not Modified` instead of the full data.
- Hint: Look up how ETags work in HTTP.

**Test:**

- Use `curl -i` to check the headers.
- Try sending `If-None-Match` with the ETag you got back.